

1 Theory

4-Bit Adder

A 4-bit adder is a digital circuit that adds two 4-bit binary numbers to produce a 4-bit sum and a carry-out. It consists of four full adders connected in series, where each full adder adds corresponding bits of the two input numbers and the carry from the previous stage. The final output is a 4-bit sum and a carry-out signal, indicating overflow if necessary.

4-Bit Subtractor

A 4-bit subtractor is a digital circuit that subtracts one 4-bit binary number from another by using the 2's complement method. It consists of four full adders, where NOT gates are used to invert each bit of the second number (the number being subtracted). By inverting the second number and adding 1 to it, subtraction is performed using addition. The circuit thus performs subtraction by adding the inverted number and a carry-in of 1.

4-Bit Adder Subtractor

A 4-bit adder-subtractor circuit can perform both addition and subtraction, based on a control signal. The circuit uses four full adders and external XOR gates. The control signal determines whether the circuit will add or subtract:

For addition, the XOR gates pass the second number unchanged to the full adders whereas for subtraction, the control signal inverts the second number by passing it through the XOR gates, performing the 2's complement method, and a carry-in of 1 is applied.

2 Implementation and Observation

2.1 4-Bit Adder

VHDL code

VHDL program of 4-bit adder was completed in 3 steps which includes 3 VHDL files.

Firstly, required code for full adder was written. Let's assume **fulladder.vhd** is the file's name.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- design of full adder

entity fulladder is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
```

```

        Cin : in STD_LOGIC;
        Sum : out STD_LOGIC;
        Cout : out STD_LOGIC);
end fulladder;

```

architecture Behavioral of fulladder is

```

begin
Sum <= A xor B xor Cin;
Cout<= (A and B) or (B and Cin) or (Cin and A);
end Behavioral;

```

Then, **fulladder.vhd** file is used repeatedly to design four full adders, join the necessary carry signals and assign required bits according to their respective significance. Let's assume name of file use includes 4-bit adder vhdl code **fourbitadder.vhd**.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- design of four bit adder

entity four_bitadder is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          --Cin : in STD_LOGIC;
          Sum : out STD_LOGIC_VECTOR (3 downto 0);
          Cout : out STD_LOGIC);
end four_bitadder;

```

architecture Behavioral of four_bitadder is

```

signal C0, C1, C2: std_logic;
component fulladder is port(
A : in std_logic;
B : in std_logic;
Cin: in std_logic;
Sum: out std_logic;
Cout: out std_logic);
end component;

```

```

begin
FA0: fulladder port map(
A => A(0),
B => B(0),
Cin => '1',
Sum => Sum(0),
Cout => C0 );

```

```

FA1: fulladder port map(
  A => A(1),
  B => B(1),
  Cin => C0,
  Sum => Sum(1),
  Cout => C1 );

```

```

FA2: fulladder port map(
  A => A(2),
  B => B(2),
  Cin => C1,
  Sum => Sum(2),
  Cout => C2 );

```

```

FA3: fulladder port map(
  A => A(3),
  B => B(3),
  Cin => C2,
  Sum => Sum(3),
  Cout => Cout );
end Behavioral;

```

Finally, a test bench file (say) **fourbitaddertb.vhd** file is created to generate virtual environment to test the design before it is implemented in hardware.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity Four_bit_adder_tb is
--  Port ( );
end Four_bit_adder_tb;

```

```

architecture Behavioral of Four_bit_adder_tb is
component Four_bit_adder is port(
  A : in STD_LOGIC_VECTOR (3 downto 0);
  B : in STD_LOGIC_VECTOR (3 downto 0);
  Sum : out STD_LOGIC_VECTOR (3 downto 0);
  Cout : out STD_LOGIC);
end component;
Signal A: STD_LOGIC_VECTOR (3 downto 0);
Signal B: STD_LOGIC_VECTOR (3 downto 0);
Signal Sum: STD_LOGIC_VECTOR (3 downto 0);
Signal Cout: STD_LOGIC;

```

```

begin
DUT:Four_bit_adder  port map( --Design Under Test
A=>A,
B=>B,
Sum=>sum,
cout=>cout);

process --Sequential block
begin
A<="0111";
B<="1000";
wait for 10ns;

A<="1001";
B<="0111";
wait;
end process;
end Behavioral;

```

RTL Schematic Diagram

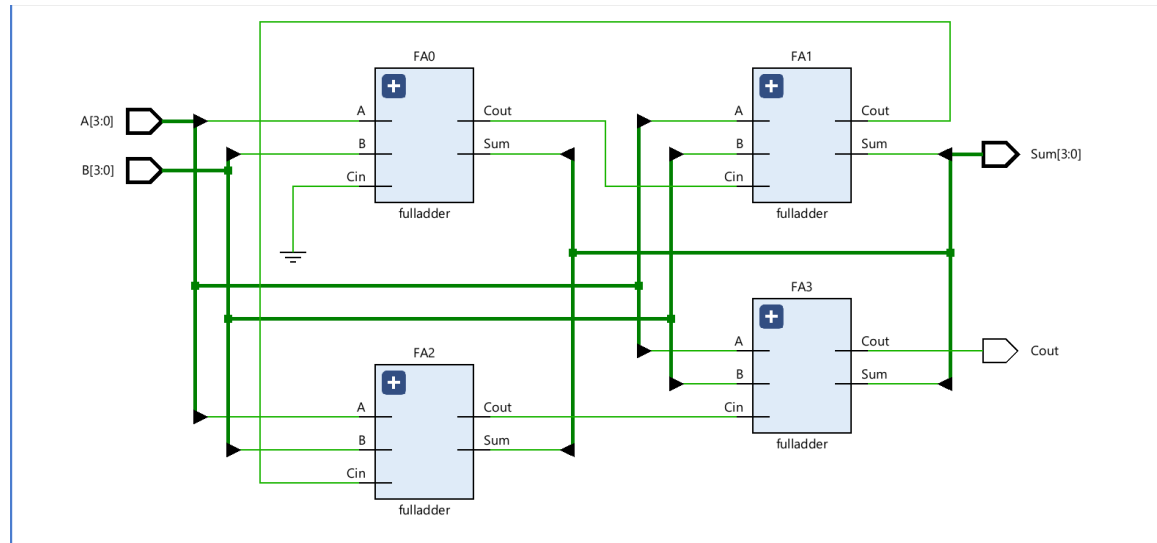


Figure 1: RTL Schematic Diagram of 4-bit adder

RTL design is used to describe the behavior of the 4-bit adder as shown in figure.

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the 4-bit adder circuit as shown in figure.

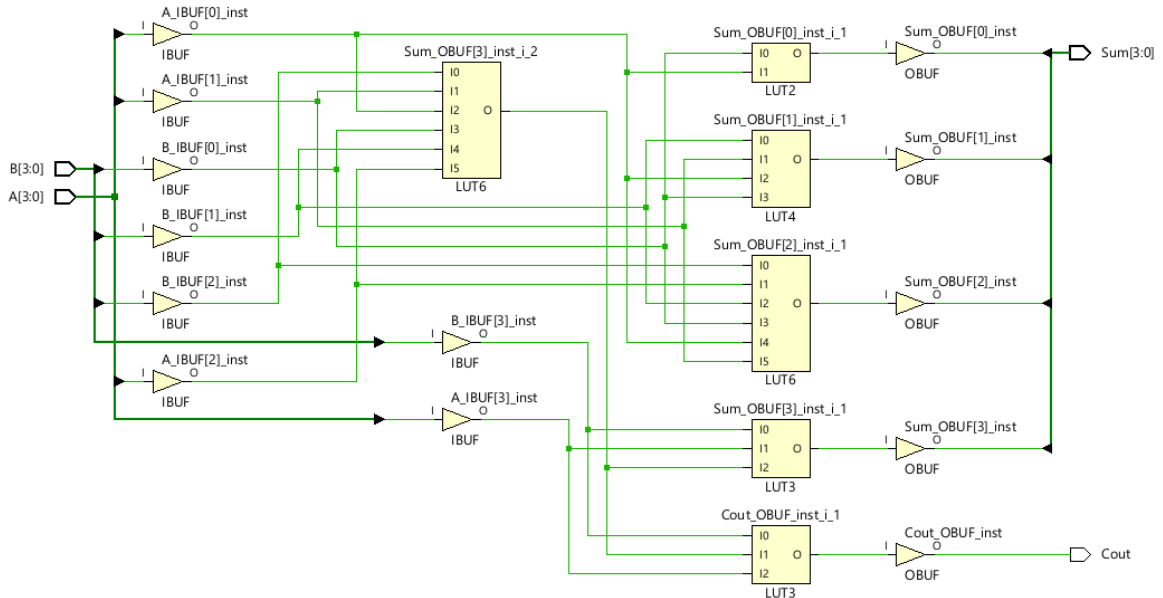


Figure 2: Synthesis Schematic Diagram of 4-bit adder

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of 4-bit adder.

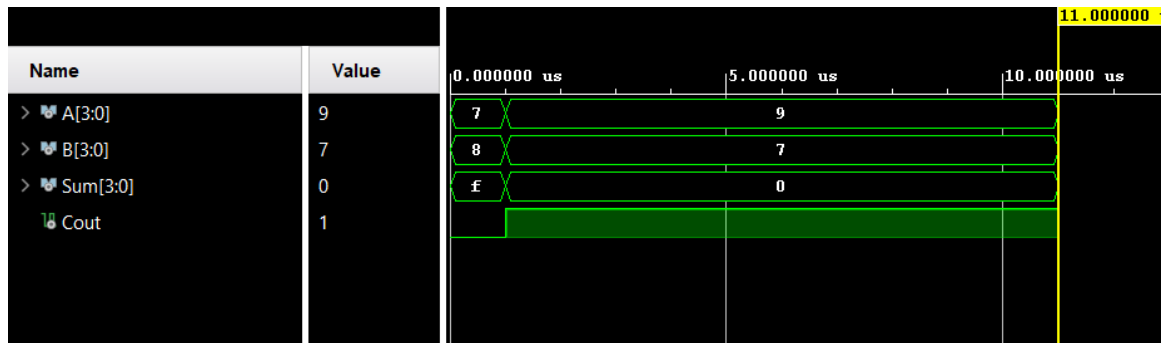


Figure 3: Timing Diagram of 4-bit adder

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure.

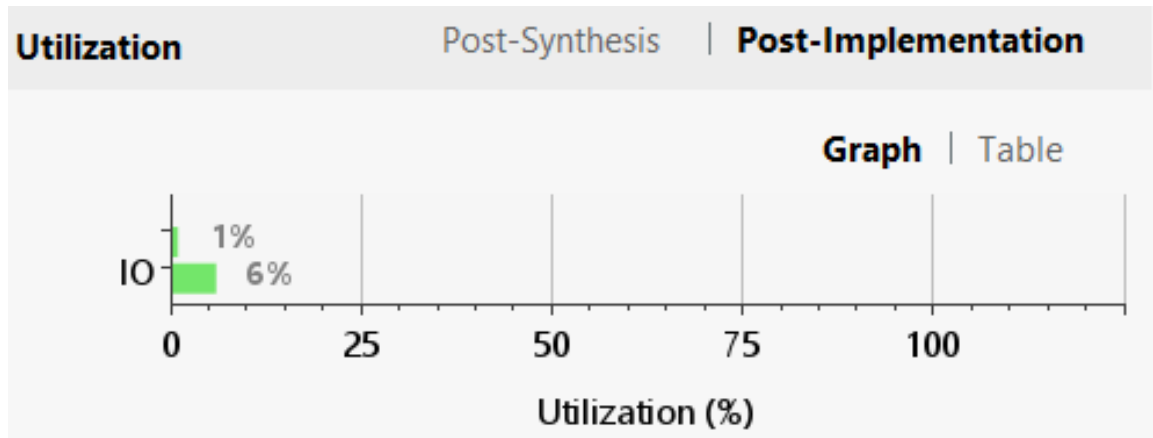


Figure 4: Device Utilization of 4-bit adder

Power Estimation

Following figure shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a 4-bit adder circuit.

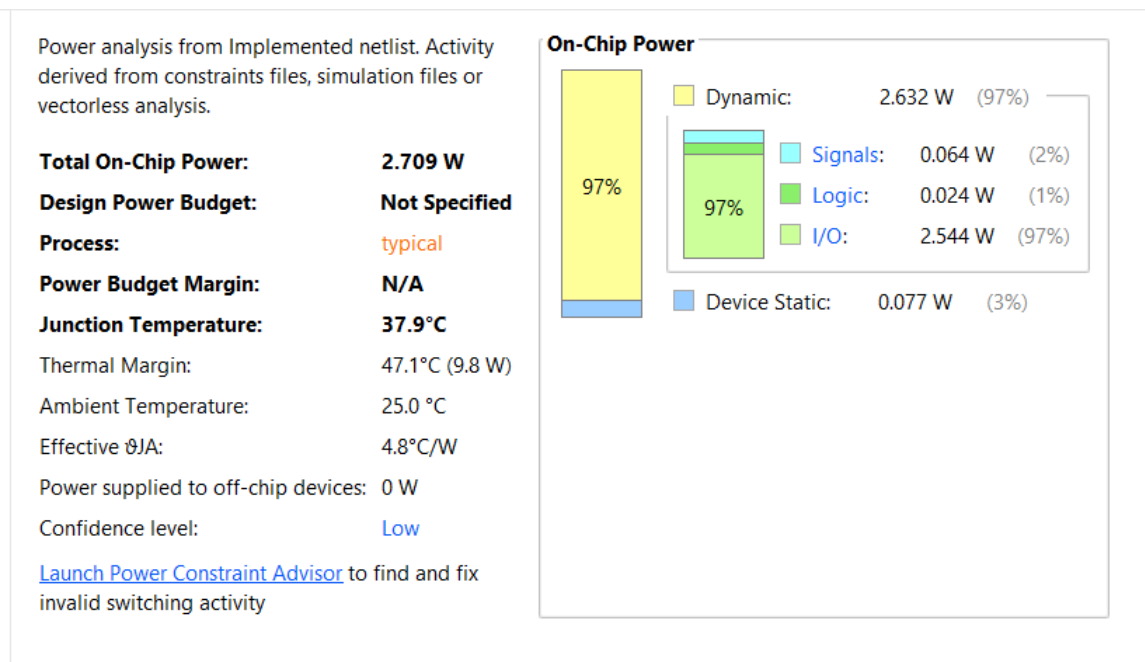


Figure 5: Power Estimation of 4-bit adder

2.2 4-Bit Subtractor

VHDL code

VHDL program of 4-bit subtractor was completed in 3 steps which includes 3 VHDL files.

Firstly, required code for full adder was written. Let's assume **fulladder.vhd** is the file's name.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- design of full adder

entity fulladder is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          Cin : in STD_LOGIC;
          Sum : out STD_LOGIC;
          Cout : out STD_LOGIC);
end fulladder;

architecture Behavioral of fulladder is

begin
    Sum <= A xor B xor Cin;
    Cout<= (A and B) or (B and Cin) or (Cin and A);
end Behavioral;
```

Then, **fulladder.vhd** file is used repeatedly to design four full adder, join the necessary carry signals and assign required bits according to their respective significance. Let's assume name of file use includes 4-bit subtractor vhd code **fourbitsubtractor.vhd**.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- design of four bit subtractor
entity four_bitsubtractor is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Diff : out STD_LOGIC_VECTOR (3 downto 0);
          Borrow : out STD_LOGIC);
end four_bitsubtractor;

architecture Behavioral of four_bitsubtractor is
    signal C0, C1, C2: STD_LOGIC;
    signal B_inverted : STD_LOGIC_VECTOR (3 downto 0);
```

```

component fulladder is
    port(
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Cin: in STD_LOGIC;
        Sum: out STD_LOGIC;
        Cout: out STD_LOGIC
    );
end component;

begin
    B_inverted <= not B;

    FA0: fulladder port map(
        A => A(0),
        B => B_inverted(0),
        Cin => '1',
        Sum => Diff(0),
        Cout => C0 );

    FA1: fulladder port map(
        A => A(1),
        B => B_inverted(1),
        Cin => C0,
        Sum => Diff(1),
        Cout => C1 );

    FA2: fulladder port map(
        A => A(2),
        B => B_inverted(2),
        Cin => C1,
        Sum => Diff(2),
        Cout => C2 );

    FA3: fulladder port map(
        A => A(3),
        B => B_inverted(3),
        Cin => C2,
        Sum => Diff(3),
        Cout => Borrow );

end Behavioral;

```

Finally, a test bench file (say) **fourbitsubtractortb.vhd** file is created to generate virtual environment to test the design before it is implemented in hardware.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity four_bitsubtractor_tb is
end four_bitsubtractor_tb;

architecture Behavioral of four_bitsubtractor_tb is
    component four_bitsubtractor is
        port(
            A : in STD_LOGIC_VECTOR (3 downto 0);
            B : in STD_LOGIC_VECTOR (3 downto 0);
            Diff : out STD_LOGIC_VECTOR (3 downto 0);
            Borrow : out STD_LOGIC
        );
    end component;

    signal A : STD_LOGIC_VECTOR (3 downto 0);
    signal B : STD_LOGIC_VECTOR (3 downto 0);
    signal Diff : STD_LOGIC_VECTOR (3 downto 0);
    signal Borrow : STD_LOGIC;

begin

    DUT: four_bitsubtractor port map(
        A => A,
        B => B,
        Diff => Diff,
        Borrow => Borrow
    );

    process
    begin

        A <= "1111";
        B <= "1000";
        wait for 10 ns;

        wait;
    end process;
```

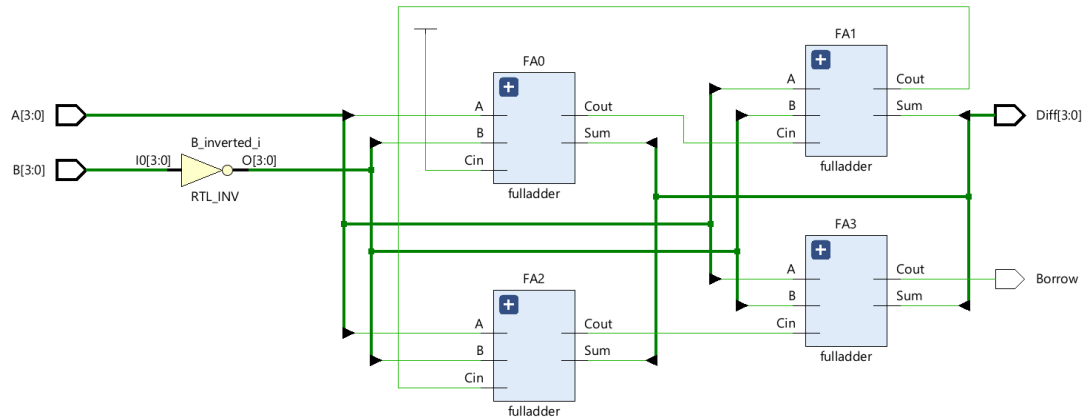


Figure 6: RTL Schematic Diagram of 4-bit subtractor

end Behavioral;

RTL Schematic Diagram

RTL design is used to describe the behavior of the 4-bit subtractor as shown in figure 6

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the 4-bit subtractor circuit as shown in figure 7.

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of 4-bit subtractor.

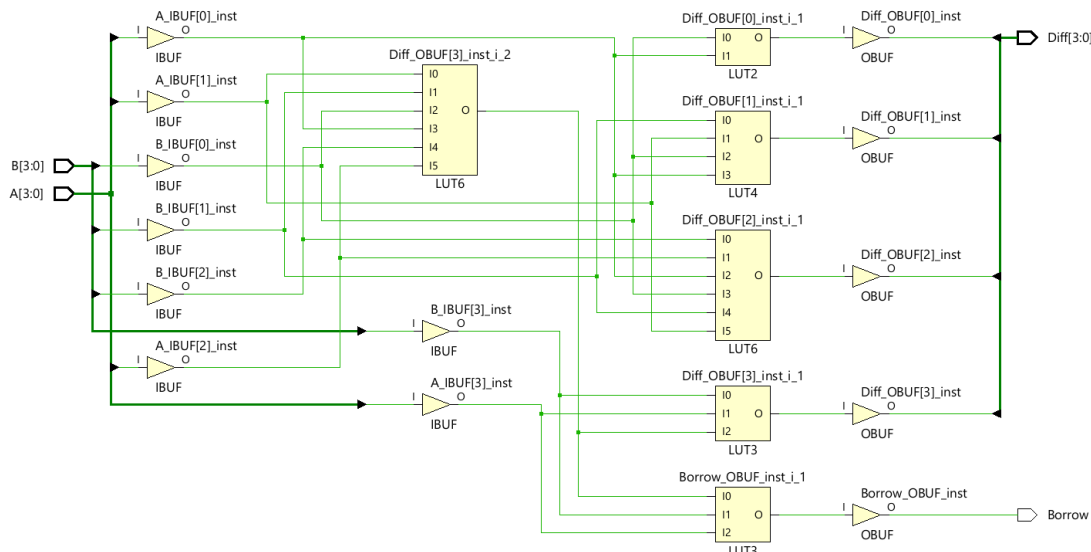
Power Estimation

Figure 9 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a 4-bit subtractor circuit.

2.3 4-Bit Adder/Subtractor

VHDL code

VHDL program of 4-bit adder/subtractor was completed in 3 steps which includes 3 VHDL files.



Firstly, required code for full adder was written. Let's assume **fulladder.vhd** is the file's name.

Then, **fulladder.vhd** file is used repeatedly to design four full adders, join the necessary carry signals and assign required bits according to their respective significance. Let's assume name of file use includes 4-bit adder/subtractor vhdl code **fourbitaddsub.vhd**.

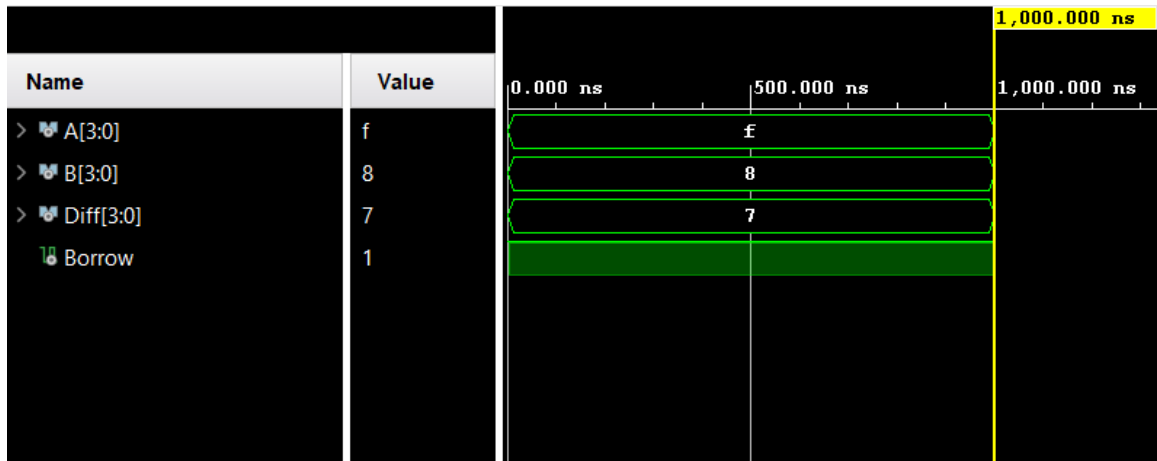


Figure 8: Timing Diagram of 4-bit subtractor

```

use IEEE.STD_LOGIC_1164.ALL;

entity four_bitaddsub is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Control : in STD_LOGIC;
          Result : out STD_LOGIC_VECTOR (3 downto 0);
          Cout : out STD_LOGIC);
end four_bitaddsub;

architecture Behavioral of four_bitaddsub is
    signal C0, C1, C2: STD_LOGIC;
    signal B_xor_control : STD_LOGIC_VECTOR (3 downto 0);

    component fulladder is
        port(
            A : in STD_LOGIC;
            B : in STD_LOGIC;
            Cin: in STD_LOGIC;
            Sum: out STD_LOGIC;
            Cout: out STD_LOGIC
        );
    end component;

begin
    B_xor_control <= B xor (Control & Control & Control & Control);

    FA0: fulladder port map(
        A => A(0),

```

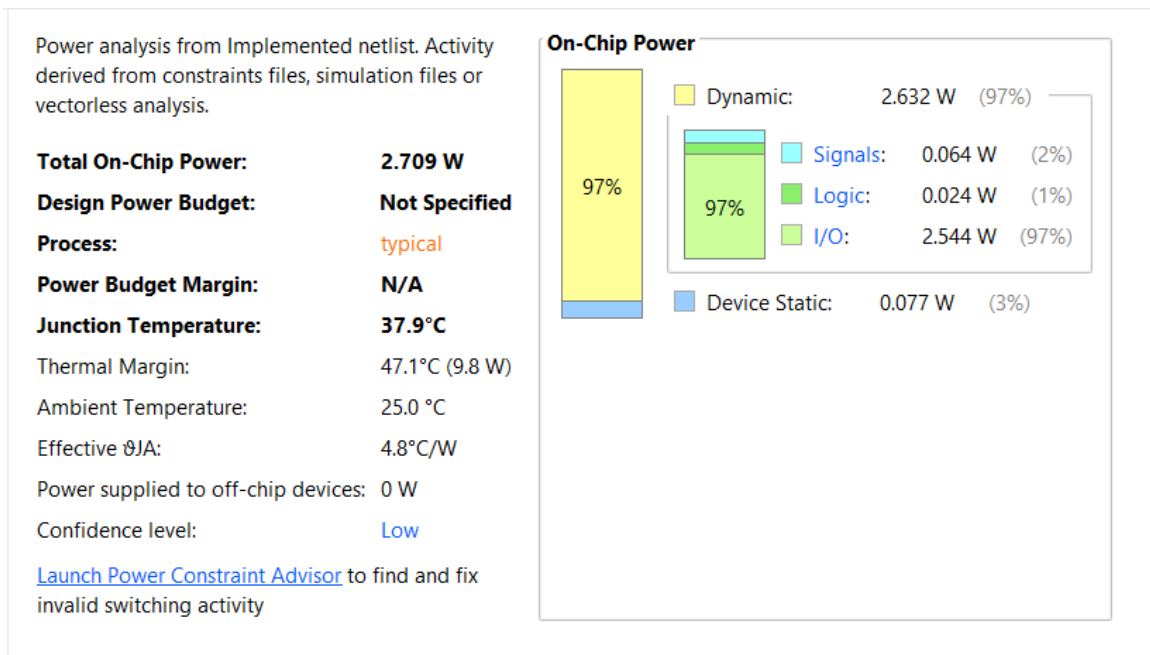


Figure 9: Power Estimation of 4-bit subtractor

```

B => B_xor_control(0),
Cin => Control,
Sum => Result(0),
Cout => C0 );

FA1: fulladder port map(
  A => A(1),
  B => B_xor_control(1),
  Cin => C0,
  Sum => Result(1),
  Cout => C1 );

FA2: fulladder port map(
  A => A(2),
  B => B_xor_control(2),
  Cin => C1,
  Sum => Result(2),
  Cout => C2 );

FA3: fulladder port map(
  A => A(3),
  B => B_xor_control(3),
  Cin => C2,
  Sum => Result(3),

```

```

        Cout => Cout );

end Behavioral;

```

Finally, a test bench file (say) **fourbitaddsubtb.vhd** file is created to generate virtual environment to test the design before it is implemented in hardware.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity four_bitaddsub_tb is
end four_bitaddsub_tb;

architecture Behavioral of four_bitaddsub_tb is
    component four_bitaddsub is
        port(
            A : in STD_LOGIC_VECTOR (3 downto 0);
            B : in STD_LOGIC_VECTOR (3 downto 0);
            Control : in STD_LOGIC;
            Result : out STD_LOGIC_VECTOR (3 downto 0);
            Cout : out STD_LOGIC
        );
    end component;

    signal A : STD_LOGIC_VECTOR (3 downto 0);
    signal B : STD_LOGIC_VECTOR (3 downto 0);
    signal Control : STD_LOGIC;
    signal Result : STD_LOGIC_VECTOR (3 downto 0);
    signal Cout : STD_LOGIC;

begin
    DUT: four_bitaddsub port map(
        A => A,
        B => B,
        Control => Control,
        Result => Result,
        Cout => Cout
    );

    process
    begin
        A <= "0111";
        B <= "0010";
        Control <= '0';
        wait;
    end process;
end Behavioral;

```

```

        A <= "0111";
        B <= "0010";
        Control <= '1';
        wait;
    end process;

end Behavioral;

```

RTL Schematic Diagram

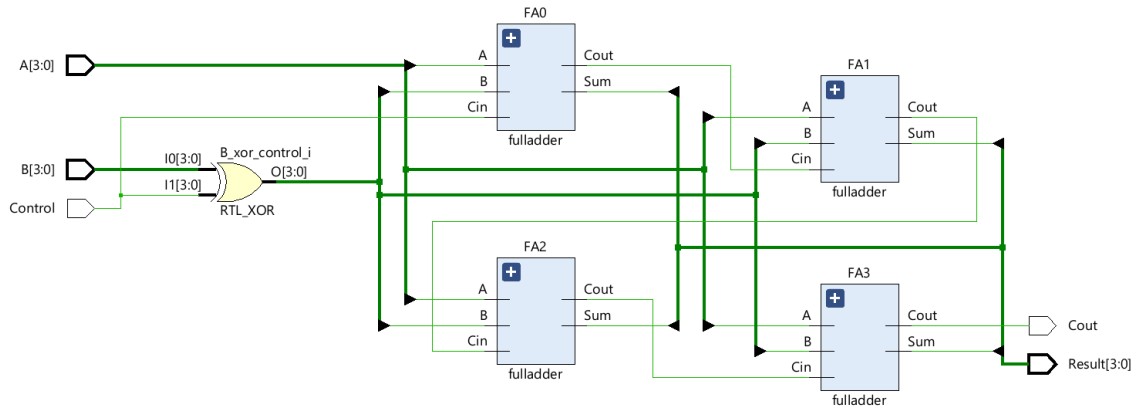


Figure 10: RTL Schematic Diagram of 4-bit adder/subtractor

RTL design is used to describe the behavior of the 4-bit adder/subtractor as shown in figure 10

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the 4-bit adder/subtractor circuit as shown in figure 11

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of 4-bit adder/subtractor as shown in figure 12.

Power Estimation

Figure 13 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a 4-bit adder/subtractor circuit.

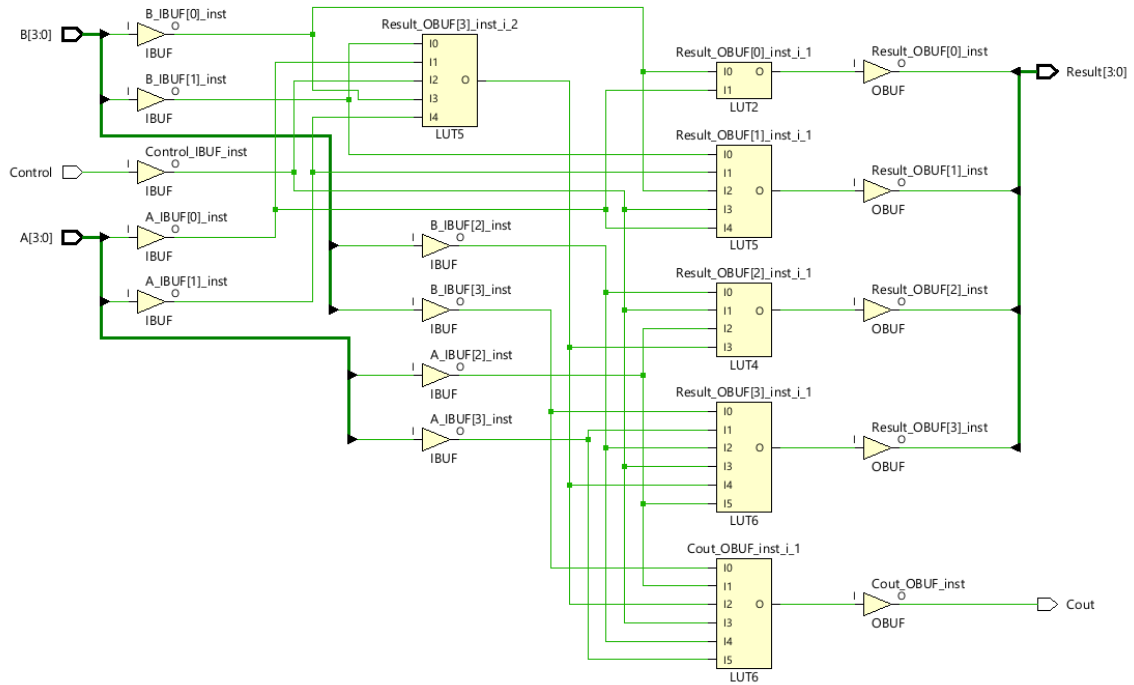


Figure 11: Synthesis Schematic Diagram of 4-bit adder/subtractor

3 Conclusion

In this lab, we implemented the VHDL code for a 4-bit adder, 4-bit subtractor and 4-bit adder on the Atrix-7 FPGA (part: xc7a50tcs324-1, package: csg324) using Vivado software. Each design was synthesized, simulated, and implemented, which confirmed their correct functionality.

We used modular approach which gave us flexibility and efficiency in the design of digital circuits possible since we used a similar full adder designed for addition and subtraction processes. Transition between addition and subtraction in the 4-bit adder-subtractor circuit by adding a control signal. We were able to implement the 2's complement method for signed arithmetic by inverting the second operand for subtraction through the use of XOR gates. By utilizing the same hardware components for both tasks, this method simplifies the design.

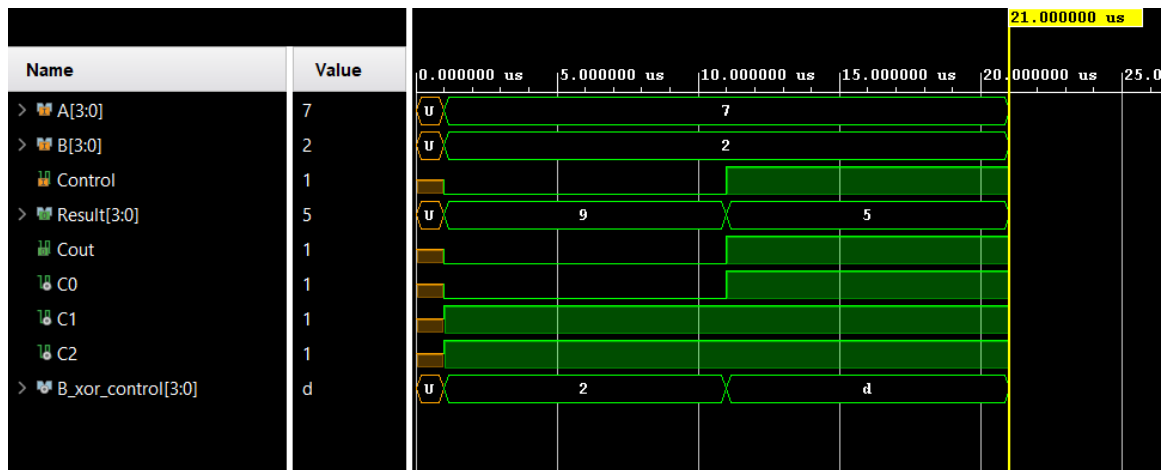


Figure 12: Timing Diagram of 4-bit adder/subtractor

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 3.05 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 39.6°C
 Thermal Margin: 45.4°C (9.4 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 4.8°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

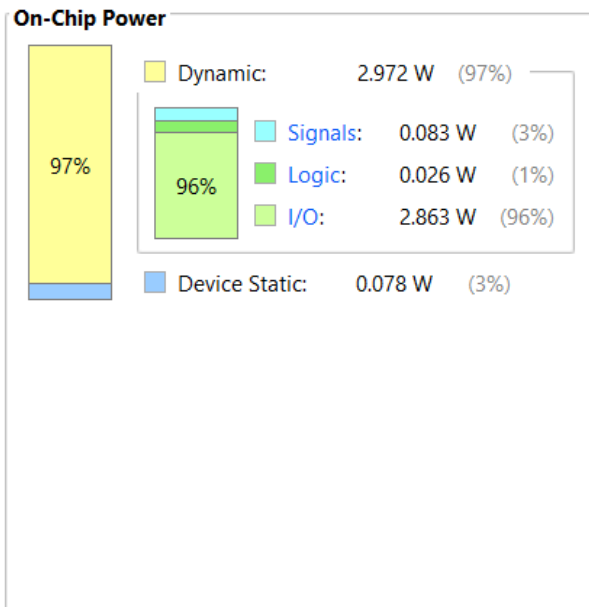


Figure 13: Power Estimation of 4-bit adder/subtractor